Proofs you can bet on: Formal Verification of Stochastic Differential Equations

Evan Misshula

July 2, 2025

What is Lean4? A modern proof assistant for formal verification.

- Proof assistant (=interactive theorem prover)

Allows users to specify theorems and proofs in a formal language

- Sometimes automates smaller proof steps (unreliable)
- Verifies proofs down to the axioms of its logical foundation
- Programming Language
- Definitions made in Lean are often executable and can be compiled into computer programs
- This is useful to automate proofs

Project was funded by Microsoft

Augmented Mathematical Intelligence (AMI)

- Empower Mathematicians working on cutting–edge mathematics
- Democratize mathematics education
- Platform for Math-AI research

On Teaching Proof

"It is dangerous to assume anything without proving it, even if everyone believes it." – JL Doob

"Writing is natures way of letting you know how sloppy your thinking is." – Leslie Lamport

"Students think proofs are like sudoku: if you just fill in the right things, it all works. But good proofs are more like jokes they have setup, structure, and punchline."
– Ravi Vakil (paraphrased)

"Proofs are mental poetry. If you recite them without understanding, they're just noise." – Francis Su

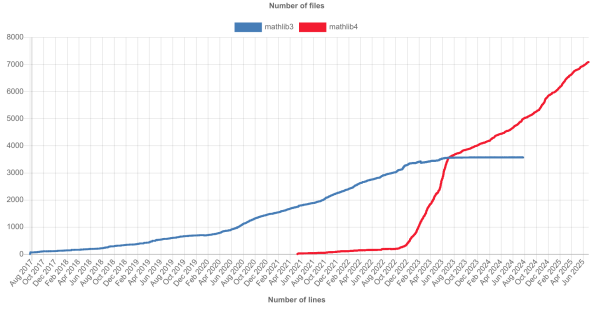
Lean is still early in its evolution

Mathlib statistics

Counts

Definitions	Theorems	Contributors
107381	220604	588

Code growth



Topic Coverage in Mathematics

- https://leanprover-community.github.io//mathlib4_docs/mathlib.html

Why use Proof Assistants?

- If it is verified and you don't understand the argument, at least you know that you are starting from something correct

Success stories

- Kepler Conjecture
- Liquid Tensor Experiment
- Erdős-Graham conjecture on unit fractions

Scholze & The Liquid Tensor Experiment

- 2020: Peter Scholze challenges formalization community: verify a key result in condensed mathematics
- Target: "Liquid Tensor Experiment" a complex statement involving higher category theory and condensed sets
- Progress:
  - Community project led by Kevin Buzzard and Johan Commelin
  - Used Lean 3 & mathlib
- Result: Successfully formalized in 2022
  - ~35,000 lines of Lean code
  - Demonstrated that even cutting-edge modern math can be formalized

"This really changes how we can think about checking math." Scholze

I brought receipts

- GitHub Repository (Lean 3):
  https://github.com/leanprover-community/liquid
- Documentation & Blog Posts:
  - Project blog: https://xenaproject.wordpress.com/2020/12/20/the-liquid-tensor-experiment/
  - Scholze's challenge (Buzzard blog): https://xenaproject.wordpress.com/2020/12/05/scholze-and-the-liquid-tensor-experiment/

Terence Tao: Verified Mathematics

- 2023: Tao collaborated on formalizing a nontrivial result in additive combinatorics
- Theorem: Every sufficiently large odd number is the sum of at most five primes (a corollary of his earlier work)
- Tools: Used Lean 4 and mathlib4
- Significance:
  - First formalized result authored by a Fields Medalist in Lean
  - Tao blogged: The Lean formalisation actually taught me new things about the proof.
- Message: Formalization is no longer just verification–its mathematical exploration

More recepiets

- GitHub Repository: This is part of general mathlib4 and Taos contribution was integrated via pull requests: https://github.com/leanprover-community/mathlib4
- Taos Blog Post on Formalization: https://terrytao.wordpress.com/2023/04/05/formal-verification-in-mathematics
- PR that mentions Taos collaboration (search via GitHub): `is_schur_triple` and related topics in additive combinatorics

Pictures

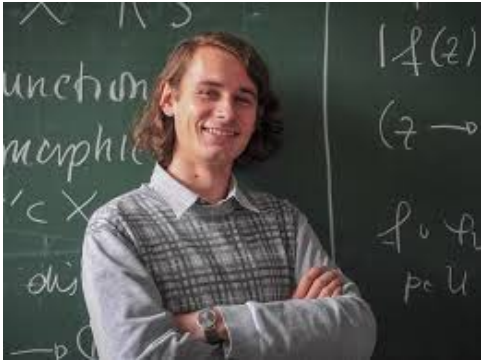


Figure: *

Peter Scholze

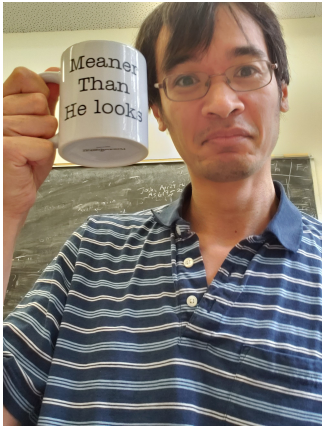


Figure: *

Terence Tao

Why SDEs? Their role in finance, physics, and engineering.

From Kings Park to Yale

- Senior year: AP Calculus AB at Kings Park High School
- Confident, top of the class math felt straightforward
- Arrived at Yale ready to major in mathematics...

  Then I walked into Math 230 with Walter Feit.

This was our math department in my high school

Culture Shock: Math 230 with Walter Feit

- Text: Calculus on Vector Spaces proof-based, abstract
- No prior exposure to – definitions, vector spaces, or proofs
- No formal proof-writing class offered in 1984

  Feit would say, Its trivial, but it requires proof.

Culture Shock: Math 230 with Walter Feit

- Text: Calculus on Vector Spaces proof-based, abstract
- No prior exposure to – definitions, vector spaces, or proofs
- No formal proof-writing class offered in 1984

  Feit would say, Its trivial, but it requires proof.
- He was co-author of what was then the longest proof ever published!
- I was completely unprepared for the rigor and abstraction

A Defining Moment

- Realized there was a deeper, more precise language to math
- Proof wasn't just a tool – it was the essence of understanding
- That shock became a turning point: I wanted to learn to think like a mathematician but I just did not have the scaffolding

A Defining Moment

- Realized there was a deeper, more precise language to math
- Proof wasn't just a tool – it was the essence of understanding
- That shock became a turning point: I wanted to learn to think like a mathematician but I just did not have the scaffolding

I didn't totally quit, I stayed math adjacent but I have never been totally comfortable knowing when I have proved something.

Formalizing Probability in Lean 4

- Probability theory is under active development in ‘mathlib4‘, ported from Lean 3
- Formalizations include:
  - Basic measure theory (-algebras, measurable spaces, integration)
  - Probability measures, expectations, laws of large numbers
- Still missing:
  - General theory of filtrations and martingales
  - Borel–Cantelli lemmas, optional stopping in full generality
- Use of ‘MeasureTheory‘, ‘ProbabilityTheory‘ modules in ‘mathlib4‘

A robust foundation is emerging–but stochastic processes are still being built.

Markov Chains and Diffusions

- Discrete-time Markov chains:
  - Basic chains and transition kernels definable via functions ✗
  - State spaces: finite or countable; transition matrices formalized
- Continuous-time processes:
  - Formalizations sparse: no full semigroup or generator framework yet
- Diffusions (e.g., Brownian motion):
  - No native stochastic calculus or Itô integration in ‘mathlib4‘ yet
  - Informal attempts exist for modeling trajectories using random walks
- Open need for stochastic differential equation (SDE) framework

Brownian Motion and Optimal Stopping

- Brownian motion: a major goal, but not yet formalized in Lean 4
  - Lean 3: partial constructions (see Liquid Tensor & random walk limits)
  - Challenge: formalizing continuous-path processes rigorously
- Stopping times and Snell envelope:
  - No full optional -algebra, martingale convergence, or Doob decomposition
- Open directions:
  - Skorokhod embedding
  - Strong Markov property
  - Reflection principle

Stopping theory is where probability meets verification–still unexplored in Lean 4.

Stochastic Control and Research Opportunities

- Stochastic control and dynamic programming remain untouched in Lean 4
  - No Hamilton–Jacobi–Bellman (HJB) formalism yet
  - No verification of Bellman's Principle or viscosity solutions
- Promising directions:
  - Formalize canonical control problems (e.g., Merton problem)
  - Verification of optimal strategies under uncertainty
  - Bridges to reinforcement learning
- Your opportunity:
  - Extend Lean 4 to handle continuous stochastic processes
  - Contribute new libraries and automation tools

Example Lean Code: \(\epsilon$-$\delta\) Definition of Limit

[go to lean]

- Defines the limit of a function ‘f‘ at a point ‘x‘
- Makes all quantifiers explicit: > 0 $> 0 \ldots$
- Used to prove continuity, derivative rules, etc.
- Based on mathlib's real analysis foundations
- GitHub: https://github.com/ImperialCollegeLondon/formalising-mathematics

  Writing this forced me to understand how all the quantifiers fit together. – Xena participant

Highlight Lean4s syntax and power.

[Demo]

Columbia's Early Legacy in Probability

Harold Robbins (1915–2001)

- Developed empirical Bayes and sequential analysis.
- *An Empirical Bayes Approach to Statistics*, 1956.
- With S. Monro: *A Stochastic Approximation Method*, 1951.

J.L. Doob (1910–2004)

- Assistant professor at Columbia (1935–1939).
- Martingale theory and rigorous foundation for stochastic processes.
- *Stochastic Processes*, Wiley, 1953.

David Siegmund and Sequential Analysis

- PhD from Columbia (1966), advised by Harold Robbins.
- Contributions to sequential testing, optimal stopping, and clinical trials.
- *Sequential Analysis: Tests and Confidence Intervals*, 1985.
- With Y.S. Chow: *Great Expectations: The Theory of Optimal Stopping*, 1971.

Our Mentor and His Impact: Ioannis Karatzas

- Faculty at Columbia since the 1980s.
- Pioneering work on stochastic control and mathematical finance.
- With S. Shreve: *Brownian Motion and Stochastic Calculus*, 1991.
- Also: *Methods of Mathematical Finance*, 1998.
- Generations of students shaped by his insight, rigor, and pedagogy.

  Being one of Dr. Karatzas's students carries weight–not just because of his rigor, clarity, and deep commitment to teaching, but because of the legacy of those who came before us. We've benefited from that legacy. Now its our turn to contribute–to help define how probability will be taught, learned, and formalized for the next generation at Columbia and beyond.

Columbia's Ongoing Influence in Probability

- Home of the *Optimal Stopping, Mathematical Finance and Probability Seminar*.
- Active areas: stochastic control, rough paths, filtering, SPDEs.
- Theoretical foundation for modern financial mathematics.
- Columbia-trained scholars now shaping the field globally.

Project Xena: Formalizing Undergraduate Mathematics

- Led by: Kevin Buzzard at Imperial College London
- Goal: Formalize the entire undergraduate math curriculum using Lean
- Focus areas:
  - Real analysis, group theory, topology, and number theory
  - Bridge the gap between research formalization and early learning
- Motivated a global wave of math formalization efforts
- Source repo: github.com/ImperialCollegeLondon/formalising-mathematics

"We are training the next generation of mathematicians to speak precisely." – Kevin Buzzard

Impact on Pedagogy

- Reinforces precision: Students learn to state theorems and definitions without ambiguity
- Immediate feedback: Errors caught by the proof assistant promote deeper understanding
- Shift in mindset:
  - From intuition-first to rigor-first reasoning
  - Encourages habits of proof construction early
- Accessibility: Could level the playing field – underrepresented students benefit from clear guidance
- Adoption: Being piloted at Imperial, Oxford, and elsewhere

It made me confront exactly what I believed a limit was – and Lean didn't let me get away with hand-waving. – Undergraduate math student

Why Lean 4 Proofs Feel 10x Harder Than LaTeX Proofs

- LaTeX proof:
  - Human-facing argument: terse, symbolic, intuitive
  - Omits implicit facts (e.g. "Fubini applies", "this is measurable")
  - Assumes context (e.g. underlying sigma-algebra, integrability, filtrations)
- Lean 4 proof:
  - Machine-facing: Every lemma must be justified or invoked explicitly
  - Requires full formal stack:
    - `Measurable f`, `Integrable f`, σ-finite-μ, `IsProbabilityMeasure`
    - Precise use of filters, types, coercions (`ENNReal`, etc.)
  - No "handwaving"if it's not in 'mathlib', you build it

Concrete Example

- Goal: Prove $\mathbb{E}[f(X)] = \int f \, d\mu$ where $X \sim \mu$, f measurable
- In LaTeX: 3 lines.
- In Lean 4: 30100 lines:
  - Construct pullback measure `X.map` μ
  - Show `f` $\circ$ `X` is integrable
  - Invoke 'lintegral' + coercions to ''
  - Possibly instantiate `AE_measurable`, `MeasurableEmbedding`, etc.

Why It Matters

- You're not just verifying the argument*you're *building the foundations.
- Like writing a compiler for measure theory from the ground up.
- Once built, others reuse it with trust.

Comparison of Lean 4, Coq, and Isabelle/HOL

- Lean 4
  - Strengths:
    - Modern metaprogramming via a real programming language (Lean itself)
    - Fast elaborator and kernel, good IDE integration (VSCode, Emacs)
    - Community-led ‘mathlib4‘: rapidly growing standard library
  - Weaknesses:
    - Less mature ecosystem than Coq/Isabelle (esp. in verified systems)
    - Fewer certified projects in software/hardware domains (for now)
    - No standard tactic language (replaces ‘Ltac‘ with metaprogramming)

Coq

- Coq
  - Strengths:
    - Established with decades of work; rich ecosystem (CompCert, VST, Fiat, etc.)
    - Powerful tactic language (‘Ltac‘), extensive formalizations
    - Strong government/institutional support (INRIA, NSF, etc.)
  - Weaknesses:
    - Slower kernel; ‘Ltac‘ can be opaque and difficult to debug
    - Less friendly for newcomers; less unified documentation
    - No native metaprogramming (but ‘MetaCoq‘ project exists)

Isabelle

- Isabelle/HOL
  - Strengths:
    - Highly automated (via ‘Sledgehammer‘, ‘Nitpick‘)
    - Mature user interface (Isabelle/jEdit)
    - Large formal libraries (e.g., Archive of Formal Proofs)
  - Weaknesses:
    - Less emphasis on dependent types (uses HOL, not CIC)
    - Syntax more rigid; harder to customize logic
    - Smaller academic user base in math vs CS

Conventional wisdom (summary)

- Lean 4: best for modern math formalization and pedagogical clarity
- Coq: best for certified software, verified compilers and systems
- Isabelle/HOL: best for automation-heavy formal proofs in CS

Project Xena and community contributions.

- Project Xena

https://www.ma.imperial.ac.uk/~buzzard/xena/ https://xenaproject.wordpress.com/

- Brownian Motion

https://remydegenne.github.io/brownian-motion/

Comparison to Linux/Python: open-source collaboration, not just contributor numbers.

Tutorial and books

- Natural Number Game:

https://adam.math.hhu.de/

- Mechanics of Proof

https://hrmacbeth.github.io/math2001

- Mathematics in Lean:

https://leanprover-community.github.io/mathematics_in_lean/

- Theorem Proving in Lean

https://leanprover.github.io/theorem_proving_in_lean4/Introduction/#Intro

- Functional Programming in Lean

https://lean-lang.org/functional_programming_in_lean/

Additional Ressources

- Zulip

https://leanprover.zulipchat.com/

- Lean community webs

Join Lean4 communities (e.g., Project Xena, workshops).

Try formalizing SDEs in Lean4.

Q&A

Lean 4 Learner Wisdom

"The student begins with the conclusion and works backward, hoping that somewhere along the way a theorem will intervene." – Math teaching folklore

"Their Lean script compiled, but no one knew why. A miracle of metavariables and automation." – From the Church of Tactic Faith

"A proof is like a story. A bad proof is like a bad story: you don't know the characters, you don't know the plot, and somehow you're supposed to feel satisfied at the end." – Teaching wisdom

"The students' tactic block consisted of all known tactics applied at random, in a desperate hope that 'simp' would save them." – Observed during office hours

"This proof is left as an exercise for the reader. The reader is left as an exercise for the type checker." – Formal methods humor